

Dossier de Projet Personnel Encadré (PPE)

Déploiement d'une solution de sauvegarde multimédia et sécurisé via Immich et Tailscale

Sommaire

1. Contexte Métier et Analyse du Besoin

- 1.1 Le Besoin : Protection et centralisation des données mobiles
- 1.2 La Problématique : Dépendance aux Clouds publics et vie privée
- 1.3 La Solution Proposée : Immich sur Micro-serveur

2. Choix Architecturaux et Technologiques

3. Phase de Réalisation et Déploiement

- 3.1 Environnement et conteneurisation (Docker)
- 3.2 Configuration du modèle de stockage NAS
- 3.3 Mise en place de l'ingestion automatique

4. Sécurisation de l'Infrastructure (Hardening)

- 4.1 Accès distant sécurisé via VPN Maillé (Tailscale)
- 4.2 Isolation des services et gestion des identités

5. Maintien en Condition Opérationnelle (MCO)

- 5.1 Stratégie de sauvegarde de la base de données et des médias
- 5.2 Supervision et résolution d'incidents

6. Bilan Personnel et Conclusion

1. Contexte Métier et Analyse du Besoin

1.1 Le Besoin : Protection et centralisation des données mobiles

L'objectif est de transformer un Raspberry Pi 5 en une plateforme capable de centraliser automatiquement les photos et vidéos issues de smartphones (iPhone/Android). Le projet doit permettre une consultation fluide et une organisation structurée des médias pour plusieurs utilisateurs (Bruno, Emilie, Stephane, Marlon, Nolan).

1.2 La Problématique : Dépendance aux Clouds publics et vie privée

Les solutions de stockage grand public (Google Photos, iCloud) posent des questions de souveraineté des données et de coûts d'abonnement croissants. L'enjeu est de retrouver le contrôle total sur sa propriété intellectuelle tout en conservant une expérience utilisateur moderne (classement par date, interface mobile).

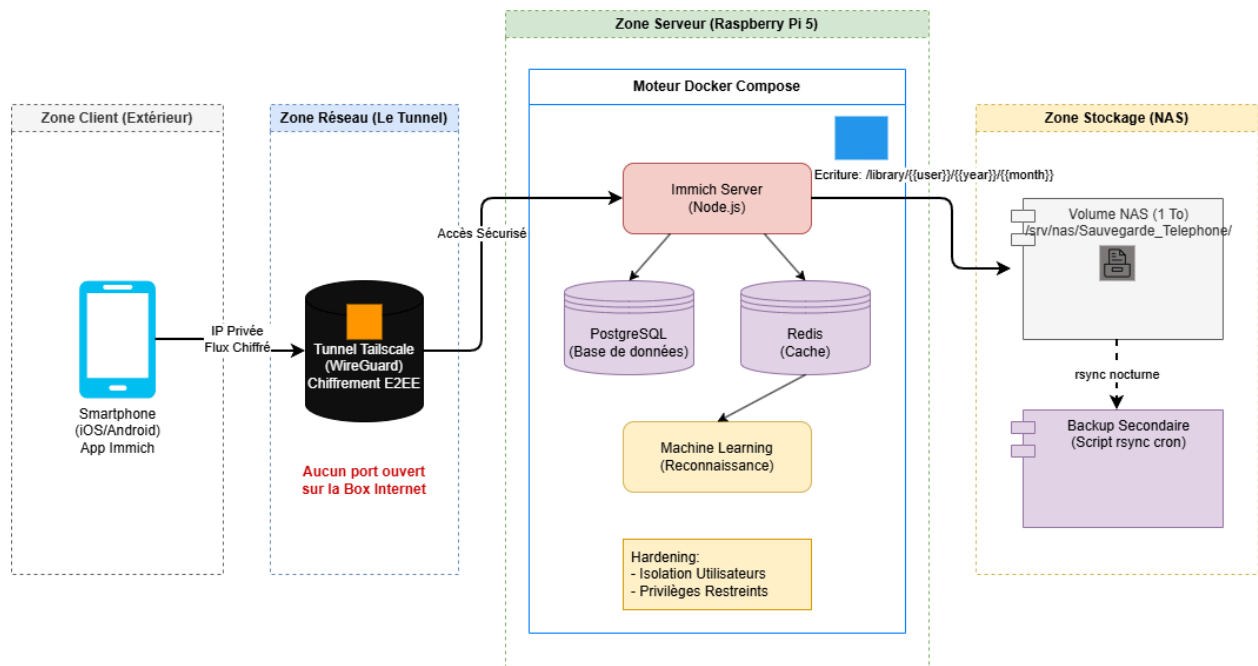
1.3 La Solution Proposée : Immich sur Micro-serveur

J'ai proposé le déploiement d'**Immich**, une solution auto-hébergée haute performance, couplée à un VPN pour garantir un accès sécurisé depuis n'importe quel réseau (4G/Wi-Fi public) sans exposition directe sur Internet.



2. Choix Architecturaux et Technologiques

- **Matériel** : Raspberry Pi 5 (4 Go RAM) pour sa puissance de calcul nécessaire au transcodage vidéo et à l'intelligence artificielle (reconnaissance d'objets).
- **Conteneurisation (Docker)** : Utilisation de Docker et Docker Compose pour isoler les micro-services d'Immich (Base de données Postgres, Redis, Micro-services de machine learning).
- **VPN (Tailscale)** : Choix d'un réseau maillé basé sur WireGuard pour chiffrer les flux de bout en bout.
- **Stockage** : Volume NAS de 1 To structuré pour accueillir les bibliothèques d'images.



3. Phase de Réalisation et Déploiement

3.1 Environnement et conteneurisation (Docker)

Le déploiement repose sur un fichier `docker-compose.yml`. Cette méthode permet de garantir la portabilité de l'application et de séparer le serveur applicatif de la base de données.

```
services:
  immich-server:
    container_name: immich_server
    image: ghcr.io/immich-app/immich-server:${IMMICH_VERSION:-release}
    volumes:
      - ${UPLOAD_LOCATION}:/data
      - /etc/localtime:/etc/localtime:ro
    env_file:
      - .env
    ports:
      - '2283:2283'
    depends_on:
      - redis
      - database
    restart: unless-stopped
    healthcheck:
      disable: false
    deploy:
      resources:
        limits:
          cpus: '2.0'
          memory: 2048M

  redis:
    container_name: immich_redis
    image: docker.io/valkey/valkey:9@sha256:3eeb09785cd61ec8e3be35f8804c8892080f3ca21934d628abc24ee4ed1698f6
    restart: unless-stopped
    healthcheck:
      test: redis-cli ping || exit 1
    deploy:
      resources:
        limits:
          cpus: '0.3'
          memory: 256M

  database:
    container_name: immich_postgres
    image: ghcr.io/immich-app/postgres:14-vectorchord0.4.3-pgvector0.2.0@sha256:bcf63357191b76a916ae5eb93464d65c07511da41e3bf7a8416db519b40b1c23
    environment:
      POSTGRES_PASSWORD: ${DB_PASSWORD}
      POSTGRES_USER: ${DB_USERNAME}
      POSTGRES_DB: ${DB_DATABASE_NAME}
      POSTGRES_INITDB_ARGS: '--data-checksums'
      # DB_STORAGE_TYPE: 'HDD'
    volumes:
      - ${DB_DATA_LOCATION}:/var/lib/postgresql/data
      shm_size: 128mb
    restart: unless-stopped
    healthcheck:
      disable: false
    deploy:
      resources:
        limits:
          cpus: '0.5'
          memory: 512M
```

3.2 Configuration du modèle de stockage NAS

J'ai configuré le "Storage Template" d'Immich pour que les fichiers soient enregistrés sur le NAS selon une structure logique et lisible : `{{y}}/{{MM}}/{{filename}}`. Ce choix facilite la maintenance

et les futures migrations de données.

Modèle

Aperçu

Limite approximative de la longueur du chemin: 70/260
bruno est l'étiquette de stockage de l'utilisateur

UPLOAD_LOCATION/library/bruno/2022/02/IMAGE_56437.jpg

Préréglage

2022/02/IMAGE_56437

Modèle *

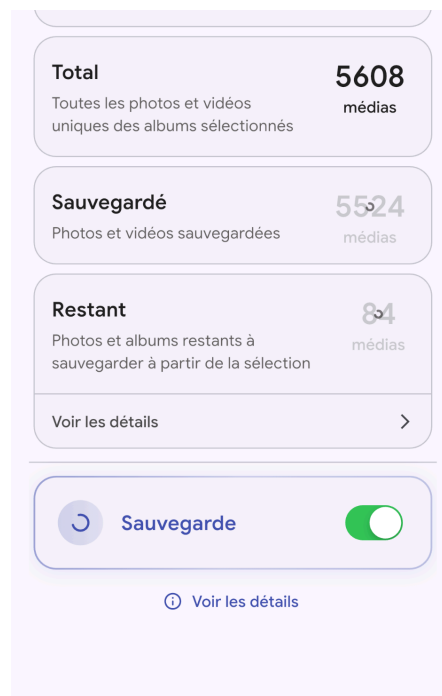
{{y}}/{{MM}}/{{filename}}

Extension

.jpg

3.3 Mise en place de l'ingestion automatique

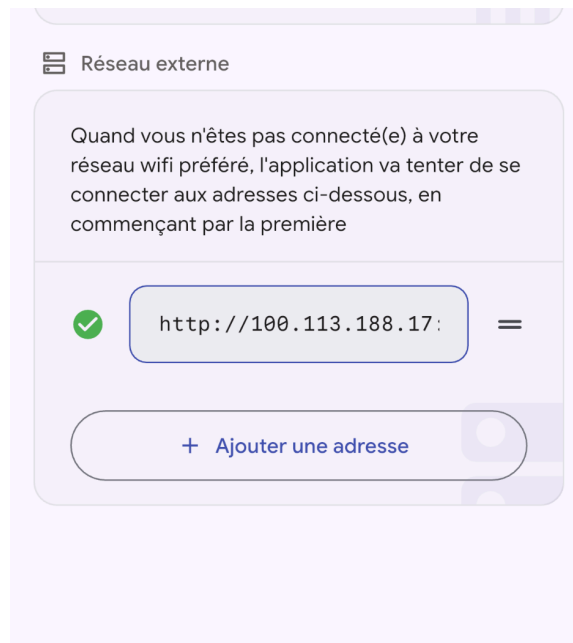
L'application mobile a été configurée pour effectuer des sauvegardes en arrière-plan dès qu'une nouvelle photo est détectée, garantissant ainsi l'absence de perte de données en cas de casse ou de vol du smartphone.



4. Sécurisation de l'Infrastructure (Hardening)

4.1 Accès distant sécurisé via VPN Maillé (Tailscale)

C'est le pilier de la sécurité du projet. Plutôt que d'ouvrir des ports sur la box Internet (ce qui exposerait l'interface d'Immich aux attaques par force brute), j'ai utilisé **Tailscale**. L'accès à la librairie photo n'est possible que pour les appareils authentifiés sur mon réseau privé virtuel.



4.2 Isolation des services et gestion des identités

Chaque utilisateur dispose de son propre compte Immich, garantissant que Marlon ne peut pas visualiser la librairie privée de Nolan. De plus, les conteneurs Docker s'exécutent avec des privilèges restreints pour limiter les risques en cas de vulnérabilité applicative.

5. Maintien en Condition Opérationnelle (MCO)

5.1 Stratégie de sauvegarde (Cron & Rsync)

Un script de sauvegarde automatisé via cron effectue chaque nuit une copie du volume /srv/nas vers un second support de stockage. J'utilise rsync pour ses capacités de synchronisation incrémentale, optimisant ainsi le temps de sauvegarde.

```
# Sauvegarde quotidienne des containers Docker
0 3 * * * rsync -av --delete /home/nolan/docker/ /home/nolan/backups/ >> /home/nolan/backups/backup.log 2>&1
nolan@RaspberryPI5-Nolan:~ $
```

5.2 Supervision et résolution d'incidents

Durant la phase de test, j'ai identifié et résolu des problèmes de chargement liés au DOM et à l'exécution des scripts. La surveillance des logs Docker (docker logs -f immich_server) est utilisée pour diagnostiquer d'éventuels échecs d'ingestion ou de transcodage.

extrait des logs :

```

[Nest] 23 - 04/16/2026, 11:11:32 AM LOG [Api:RouterExplorer] Mapped {/api/shared-links/:id, PATCH} route
[Nest] 23 - 04/16/2026, 11:11:32 AM LOG [Api:RouterExplorer] Mapped {/api/shared-links/:id, DELETE} route
[Nest] 23 - 04/16/2026, 11:11:32 AM LOG [Api:RouterExplorer] Mapped {/api/shared-links/:id/assets, PUT} route
[Nest] 23 - 04/16/2026, 11:11:32 AM LOG [Api:RouterExplorer] Mapped {/api/shared-links/:id/assets, DELETE} route
[Nest] 23 - 04/16/2026, 11:11:32 AM LOG [Api:RoutesResolver] StackController {/api/stacks}:
[Nest] 23 - 04/16/2026, 11:11:32 AM LOG [Api:RouterExplorer] Mapped {/api/stacks, GET} route
[Nest] 23 - 04/16/2026, 11:11:32 AM LOG [Api:RouterExplorer] Mapped {/api/stacks, POST} route
[Nest] 23 - 04/16/2026, 11:11:32 AM LOG [Api:RouterExplorer] Mapped {/api/stacks, DELETE} route
[Nest] 23 - 04/16/2026, 11:11:32 AM LOG [Api:RouterExplorer] Mapped {/api/stacks/:id, GET} route
[Nest] 23 - 04/16/2026, 11:11:32 AM LOG [Api:RouterExplorer] Mapped {/api/stacks/:id, PUT} route
[Nest] 23 - 04/16/2026, 11:11:32 AM LOG [Api:RouterExplorer] Mapped {/api/stacks/:id, DELETE} route
[Nest] 23 - 04/16/2026, 11:11:32 AM LOG [Api:RouterExplorer] Mapped {/api/stacks/:id/assets/:assetId, DELETE} route
[Nest] 23 - 04/16/2026, 11:11:32 AM LOG [Api:RoutesResolver] SyncController {/api/sync}:
[Nest] 23 - 04/16/2026, 11:11:32 AM LOG [Api:RouterExplorer] Mapped {/api/sync/full-sync, POST} route
[Nest] 23 - 04/16/2026, 11:11:32 AM LOG [Api:RouterExplorer] Mapped {/api/sync/delta-sync, POST} route
[Nest] 23 - 04/16/2026, 11:11:32 AM LOG [Api:RouterExplorer] Mapped {/api/sync/stream, POST} route
[Nest] 23 - 04/16/2026, 11:11:32 AM LOG [Api:RouterExplorer] Mapped {/api/sync/ack, GET} route
[Nest] 23 - 04/16/2026, 11:11:32 AM LOG [Api:RouterExplorer] Mapped {/api/sync/ack, POST} route
[Nest] 23 - 04/16/2026, 11:11:32 AM LOG [Api:RouterExplorer] Mapped {/api/sync/ack, DELETE} route
[Nest] 23 - 04/16/2026, 11:11:32 AM LOG [Api:RoutesResolver] SystemConfigController {/api/system-config}:
[Nest] 23 - 04/16/2026, 11:11:32 AM LOG [Api:RouterExplorer] Mapped {/api/system-config, GET} route
[Nest] 23 - 04/16/2026, 11:11:32 AM LOG [Api:RouterExplorer] Mapped {/api/system-config/defaults, GET} route
[Nest] 23 - 04/16/2026, 11:11:32 AM LOG [Api:RouterExplorer] Mapped {/api/system-config, PUT} route
[Nest] 23 - 04/16/2026, 11:11:32 AM LOG [Api:RouterExplorer] Mapped {/api/system-config/storage-template-options, GET} route
[Nest] 23 - 04/16/2026, 11:11:32 AM LOG [Api:RoutesResolver] SystemMetadataController {/api/system-metadata}:
[Nest] 23 - 04/16/2026, 11:11:32 AM LOG [Api:RouterExplorer] Mapped {/api/system-metadata/admin-onboarding, GET} route
[Nest] 23 - 04/16/2026, 11:11:32 AM LOG [Api:RouterExplorer] Mapped {/api/system-metadata/admin-onboarding, POST} route
[Nest] 23 - 04/16/2026, 11:11:32 AM LOG [Api:RouterExplorer] Mapped {/api/system-metadata/reverse-geocoding-state, GET} route
[Nest] 23 - 04/16/2026, 11:11:32 AM LOG [Api:RouterExplorer] Mapped {/api/system-metadata/version-check-state, GET} route
[Nest] 23 - 04/16/2026, 11:11:32 AM LOG [Api:RoutesResolver] TagController {/api/tags}:
[Nest] 23 - 04/16/2026, 11:11:32 AM LOG [Api:RouterExplorer] Mapped {/api/tags, POST} route
[Nest] 23 - 04/16/2026, 11:11:32 AM LOG [Api:RouterExplorer] Mapped {/api/tags, GET} route
[Nest] 23 - 04/16/2026, 11:11:32 AM LOG [Api:RouterExplorer] Mapped {/api/tags, PUT} route
[Nest] 23 - 04/16/2026, 11:11:32 AM LOG [Api:RouterExplorer] Mapped {/api/tags/assets, PUT} route
[Nest] 23 - 04/16/2026, 11:11:32 AM LOG [Api:RouterExplorer] Mapped {/api/tags/:id, GET} route
[Nest] 23 - 04/16/2026, 11:11:32 AM LOG [Api:RouterExplorer] Mapped {/api/tags/:id, PUT} route
[Nest] 23 - 04/16/2026, 11:11:32 AM LOG [Api:RouterExplorer] Mapped {/api/tags/:id, DELETE} route
[Nest] 23 - 04/16/2026, 11:11:32 AM LOG [Api:RouterExplorer] Mapped {/api/tags/:id/assets, PUT} route
[Nest] 23 - 04/16/2026, 11:11:32 AM LOG [Api:RouterExplorer] Mapped {/api/tags/:id/assets, DELETE} route
[Nest] 23 - 04/16/2026, 11:11:32 AM LOG [Api:RoutesResolver] TimelineController {/api/timeline}:
[Nest] 23 - 04/16/2026, 11:11:32 AM LOG [Api:RouterExplorer] Mapped {/api/timeline/buckets, GET} route

```

6. Bilan Personnel et Conclusion

Ce projet PPE sur Immich et Tailscale complète mon expertise NAS en y ajoutant une couche applicative complexe et une gestion moderne de la mobilité. J'ai pu valider des compétences essentielles du référentiel SISR :

- **Administration Linux et Docker** en environnement de production.
- **Sécurisation des accès distants** via des technologies VPN innovantes.
- **Gestion de projet** : passage du besoin utilisateur à une infrastructure fonctionnelle et souveraine.

La solution est aujourd'hui totalement opérationnelle, offrant une alternative crédible et sécurisée aux services de Cloud public.